

CNN 이미지 분류 실무 복습 가이드

1. 개발 환경 설정

1.1 환경 선택

로컬 환경 (Anaconda)

- Anaconda3-2023.03-1-Windows-x86_64.exe 설치
- GPU 사용시: `pip install tensorflow==2.10` (Windows 10)
- CPU만 사용시: `pip install tensorflow`

클라우드 환경 (Google Colab 권장)

- Chrome 브라우저 → Google Drive → 신규 → Google Colaboratory
- 한글 폰트 설정:

```
bash
```

```
!sudo apt-get install -y fonts-nanum
```

```
!sudo fc-cache -fv
```

```
!rm ~/.cache/matplotlib -rf
```

1.2 주요 단축키

- `Shift + Enter`: 셀 실행 후 다음 셀로 이동
- `Ctrl + Enter`: 현재 셀만 실행
- `Alt + Enter`: 현재 셀 실행 후 새 셀 추가

2. TensorFlow Keras 핵심 개념

2.1 모델 생성 방법 비교

Sequential API (간단한 모델)

```
python
```

```
from tensorflow.keras import Sequential
```

```
from tensorflow.keras.layers import Input, Dense
```

```
model = Sequential()
```

```
model.add(Input(shape=4))
```

```
model.add(Dense(50, activation='relu'))
```

```
model.add(Dense(3, activation='softmax'))
```

Functional API (복잡한 모델)

```
python

from tensorflow.keras import Model
from tensorflow.keras.layers import Input, Dense, concatenate

input_layer = Input(shape=(4,))
dense1 = Dense(50, activation='relu')(input_layer)
dense2 = Dense(80, activation='relu')(input_layer)
concat = concatenate([dense1, dense2])
output = Dense(3, activation='softmax')(concat)
model = Model(inputs=input_layer, outputs=output)
```

2.2 Keras vs PyTorch 차이점

기능	Keras	PyTorch
모델 정의	Sequential/Functional API	nn.Module 클래스 상속
레이어 추가	model.add()	__init__에서 정의
순전파	자동 처리	forward() 메서드 직접 구현
모델 컴파일	model.compile()	없음 (수동 구현)
훈련	model.fit()	훈련 루프 직접 작성

3. CNN 핵심 개념

3.1 합성곱(Convolution) 연산

- 목적: 이미지의 특징을 추출
- 필터: 2D 배열로 이미지의 특정 패턴을 감지
- 연산: 필터와 이미지 영역의 요소별 곱셈 후 합계

3.2 주요 필터 종류

- Sharpen: 이미지 선명도 향상
- Blur: 이미지 흐림 효과 (Gaussian, Box blur)
- Edge Detection: 경계선 검출 (Sobel, Prewitt, Roberts)

3.3 풀링(Pooling)

- 목적: 차원 축소 및 계산량 감소
- Max Pooling: 영역에서 최댓값 선택 (가장 많이 사용)
- Average Pooling: 영역의 평균값 계산
- 일반적 설정: 2x2 크기, stride=2

3.4 스트라이드(Stride)와 패딩(Padding)

- **Stride:** 필터가 이동하는 간격 (일반적으로 1)
- **Zero Padding:** 이미지 경계에 0을 추가하여 크기 유지
- **Keras 설정:** `padding='same'` (제로패딩), `padding='valid'` (패딩 없음)

4. 실무 CNN 모델 구현

4.1 MNIST 손글씨 숫자 인식 모델

데이터 준비

```
python
```

```
from tensorflow.keras.datasets import mnist
```

```
# 데이터 로드
```

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
# 전처리
```

```
X_train = X_train.reshape(-1, 28, 28, 1) / 255.0 # 정규화
```

```
X_test = X_test.reshape(-1, 28, 28, 1) / 255.0
```

```
# 데이터 형태: (샘플수, 높이, 폭, 채널) - Channel Last 방식
```

모델 구성

python

```
from tensorflow.keras import Sequential, layers
```

```
model = Sequential([
    layers.Input(shape=(28,28,1)),

    # 첫 번째 합성곱 블록
    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Dropout(0.1),

    # 두 번째 합성곱 블록
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Dropout(0.25),

    # 완전연결층
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
```

모델 훈련

python

```
# 컴파일
model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# 훈련
model.fit(X_train, y_train,
        batch_size=128,
        epochs=10,
        verbose=1)

# 평가
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f'Test accuracy: {accuracy:.4f}')
```

4.2 주요 하이퍼파라미터

모델 구조

- Conv2D 필터 개수: $32 \rightarrow 64 \rightarrow 128$ (점진적 증가)
- 필터 크기: 3×3 (가장 일반적)
- 활성화 함수: ReLU (은닉층), Softmax (출력층)

정규화 기법

- Dropout: $0.1 \rightarrow 0.25 \rightarrow 0.5$ (점진적 증가)
- 목적: 과적합 방지

훈련 설정

- 배치 크기: 128
- 에폭: 10회
- 옵티마이저: Adam
- 손실 함수: `sparse_categorical_crossentropy`

5. 데이터 형태 이해

5.1 Channel First vs Channel Last

- **TensorFlow/Keras**: Channel Last (높이, 폭, 채널)
- **PyTorch**: Channel First (채널, 높이, 폭)

예시

- 컬러 이미지 ($28 \times 28 \times 3$)
 - Channel Last: (28, 28, 3)
 - Channel First: (3, 28, 28)

5.2 One-Hot Encoding

- 목적: 범주형 데이터를 수치형으로 변환
- 방법: 해당 클래스만 1, 나머지는 0
- Keras 팁: `sparse_categorical_crossentropy` 사용시 원-핫 인코딩 불필요

6. 실무 팁

6.1 모델 성능 향상 방법

1. 데이터 증강: 회전, 이동, 확대/축소
2. 배치 정규화: 각 층의 입력을 정규화
3. 학습률 스케줄링: 학습 진행에 따라 학습률 조정

4. 앙상블: 여러 모델의 예측 결합

6.2 디버깅 체크리스트

- 데이터 형태 확인: `.shape` 출력
- 정규화 확인: 픽셀 값이 0-1 범위인지
- 레이블 형태: 원-핫 인코딩 여부
- 손실 함수: 문제 유형에 맞는 선택

6.3 주요 에러 해결

- 차원 불일치: `reshape()` 사용하여 형태 조정
- 메모리 부족: 배치 크기 감소
- 과적합: Dropout 비율 증가, 데이터 증강

7. 다음 단계 학습 방향

1. 고급 **CNN 아키텍처**: ResNet, VGG, Inception
2. **전이 학습**: 사전 훈련된 모델 활용
3. **객체 탐지**: YOLO, R-CNN
4. **이미지 분할**: U-Net, Mask R-CNN
5. **생성 모델**: GAN, VAE

참고: 이 문서는 실무 적용을 위한 핵심 내용을 요약한 것입니다. 더 자세한 이론적 배경은 관련 논문과 문서를 참조하시기 바랍니다.